ARAB ACADEMY FOR SCIENCE & TECHNOLOGY & MARITIME TRANSPORT
COLLEGE OF ENGINEERING & TECHNOLOGY

Course	**CC410 - ECE3502 Systems Programming**
Lecturer	Dr. Sherine Nagy Saleh – Dr. Noha Seddik Abdelsalam
TA	Eng. Aya Barakat– Eng. Yara ElMekky – Eng. Mahmoud Haissam

MODIFIED SIC/XE ASSEMBLER PROJECT

Create a program that emulates a modified version of the SIC/XE machine. This simulation should include both the first and second passes of assembly, and generate HTME records for the modified SIC/XE architecture.

Introduction to modi-SIC/XE

- Consists of the original SIC/XE instruction set.
- Utilizes the same data management techniques with BYTE, WORD, RESB, and RESW.
- Preserves the existing addressing modes found in SIC/XE.
- Adds support for **Blocks, with slight modifications**, enabling the program to be divided into up to five blocks:

1. DEFAULT block, containing executable instructions in formats 1 and 2.
2. DEFAULTB block, encompassing executable instructions in formats 3, 4.
3. CDATA block, designated for smaller-sized data.
4. CBLKS block, intended for larger memory blocks.
5. POOL block, a special assembler-managed block used to store pooled data elements generated during assembly. These include operand values (e.g., &X'10' in LDA &X'10') that are identified during parsing and assigned memory in the POOL block.

The assembler must start managing the POOL block upon encountering the first operand prefixed with **&.** Whenever such an operand appears in an instruction:

- It is added to the **poolTable** (if not already present).
- It is assigned an address within the **POOL block**.
- The **POOL block location counter** is incremented accordingly.

All operand values prefixed with & are therefore stored in the POOL block, regardless of the currently active block. The POOL block is built incrementally as these operands are encountered during Pass 1.

- The position of the **POOL block** within the final memory layout is determined by the point at which the first & operand is encountered.
- The POOL block is managed entirely by the assembler and is not intended to contain regular instructions or user-defined data.

Notes

- Up to five distinct memory blocks may be utilized to segment the program logically: DEFAULT, DEFAULTB, CDATA, CBLKS and POOLBLK.
- Your code should also include error handling to detect
    - **Unidentified Block Names:** The assembler should validate block names used in the

assembly code against the predefined block names. Any reference to blocks outside of the specified names (DEFAULT, DEFAULTB, CDATA, CBLKS) should trigger an error, indicating an unrecognized block name.

- **Unidentified Symbols:** Proper error messages should be generated for symbols that are referenced but not defined or are outside the scope of their declaration. This check ensures all symbols used in the program are correctly declared and accessible within their defined blocks.
- **POOLVAR error;** when generating object code, if a referenced operand in the POOL block cannot be addressed using **PC-relative** or **Base-relative** addressing, an error should be raised.

Modi-SIC/XE Implementation Details

It takes as an input a text file (in.txt) that contains an assembly program written in modi-SIC/XE assembly language containing comments, number of lines.

Generating Intermediate File

Develop a program that processes a text file (in.txt), which includes an assembly program written in the modified SIC/XE assembly language. This file can also include comments. The primary task is to generate an intermediate file.

Pass 1 Objective: Process the initial input file to determine the memory allocation and create a symbol table.

- **Task:** Read the provided input file ("in.txt" before cleaning to "intermediate.txt") to compute and track the location counter (LC) for each program line. Simultaneously, identify and catalog symbols (labels, constants, etc.) to generate a symbol table.
- **Outputs:**
  - **symbTable.txt:** This file should include the symbol table with entries for all labels or symbols in the program mapped to their respective LC values.
  - **PoolTable.**txt**:** This file should include the pool table with entries for all pooled variables in the program mapped to their respective LC values.

Pass 2 Objective: Convert assembly instructions into machine code and format the output for program loading and execution.

- **Task:** Generate object code for each line and compile the output into the HTME format (Header, Text, Modification, End records).
- **Outputs:**
  - **out_pass2.txt:** This file should contain the object code for each line of the program, corresponding to the machine-readable instructions generated from the assembly code.
  - **HTME.txt:** This file should organize the object code into the HTME format, necessary for loading the program into memory and execution, detailing segments for initialization, execution code, modifications for relocatability, and the end record signaling the end of the object program.

In addition, if any error is encountered during Pass 1 or Pass 2, the assembler must immediately stop execution and generate a file named error.txt. This file should include the **error name** (as defined in the error handling section above) and the **program counter (PC)** value at which the error occurred.

Evaluation

Evaluation will be conducted automatically using testing scripts. The full program must be executable using a single command. The program should accept an input file named `in.txt` and generate all required output files exactly as specified above, as these will be used for validation.

For example, a Python implementation should run as:

```
python3 assembler.py in.txt
```

Failure to execute the full program in a single command, or to produce the required output files with the correct names and format, will result in failing this test case and a deduction of marks.

BONUS – 2 extra marks

Create a memory visualization interface. Display memory in hexadecimal format with proper addressing. Implement loading functionality to place object code in memory

Memory Layout:

- Byte-addressable memory
- Hexadecimal display format
- 16 columns per row (0–F)
- Program starts at location 0x000000

GUI Layout Example:

```
Address: 0 1 2 3 4 5 6 7 8 9 A B C D E F

000000: 17 10 00 48 65 6C 6C 6F 00 00 00 00 00 00 00 00
000010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
000020: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Modi-SIC/XE Assessment Details

A maximum of 3 students per group is allowed, each team member will be assessed for his individual work as well as for the group work. group members should agree on the programming languages and code structures to be used. The table below shows the tasks required from each team member.

Deadline for submission SAT 9th of May at 12PM – HARD DEADLINE

A 1 mark bonus will be granted for all team members, submitting the project before the 4th of MAY.